EXPERIMENT: 1

Aim: Implement multilayer perceptron algorithm for MNIST Hand written Digit Classification.

Description : One of your assignments is to implement a backpropagation algorithm on a simple MLP architecture. Now, based on that simple BPNN architecture, develop a new model of BPNN capable of tackling a general digit recognition task. Once again, please employ digit datasets taken from sklearn import datasets and split into training (80%) and validation (20%) datasets. Since the size of the sample image is 8 x 8 pixels, so the architecture of your new BPNN model has to be adjusted accordingly. Your python notebook must include the computation of

(A) training accuracy and validation accuracy curves, and

(B) performance analysis of the model based on the associated confusion matrix.

Program:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
gray_scale = 255
x_train = x_train.astype('float32') / gray_scale
x_test = x_test.astype('float32') / gray_scale
print("Feature matrix (x_train):", x_train.shape)
print("Target matrix (y_train):", y_train.shape)
print("Feature matrix (x_test):", x_test.shape)
print("Target matrix (y_test):", y_test.shape)
```

Output:

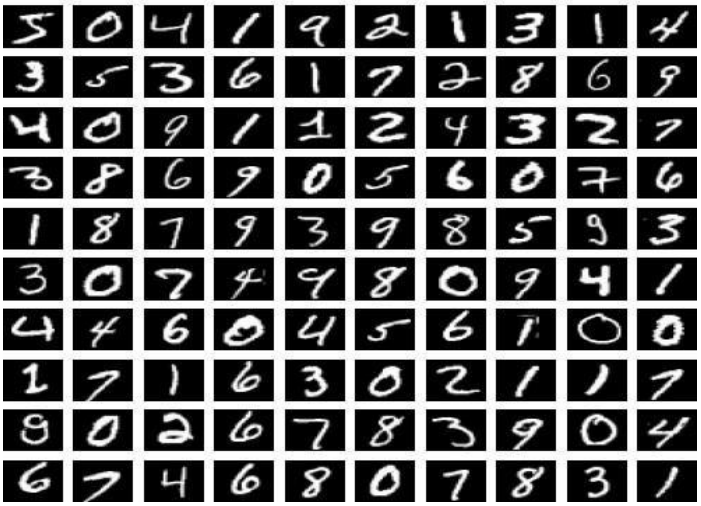
```
Feature matrix (x_train): (60000, 28, 28)
Target matrix (y_train): (60000,)
Feature matrix (x_test): (10000, 28, 28)
Target matrix (y_test): (10000,)
```

Program:

```
fig, ax = plt.subplots(10, 10)
k = 0
for i in range(10):
    for j in range(10):
    ax[i][j].imshow(x_train[k].reshape(28, 28), aspect='auto')
    k += 1
plt.show()
```

Output:

Sample Images from MNIST Dataset



Program:

```
model = Sequential([
Flatten(input_shape=(28, 28)),
Dense(256, activation='sigmoid'),
Dense(128, activation='sigmoid'),
Dense(10, activation='softmax'),
])
model.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])
mod = model.fit(x_train, y_train, epochs=10,
        batch_size=2000,
        validation_split=0.2)
print(mod)
```

Output:

```
Epoch 1/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 73ms/step - accuracy: 0.9232 - loss: 0.2767 - val_accuracy: 0.9276 - val_loss: 0.2544
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 60ms/step - accuracy: 0.9282 - loss: 0.2558 - val_accuracy: 0.9312 - val_loss: 0.2439
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.9317 - loss: 0.2422 - val_accuracy: 0.9334 - val_loss: 0.2333
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 77ms/step - accuracy: 0.9332 - loss: 0.2334 - val_accuracy: 0.9362 - val_loss: 0.2244
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 87ms/step - accuracy: 0.9373 - loss: 0.2242 - val_accuracy: 0.9385 - val_loss: 0.2164
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 59ms/step - accuracy: 0.9371 - loss: 0.2170 - val_accuracy: 0.9400 - val_loss: 0.2098
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.9387 - loss: 0.2093 - val_accuracy: 0.9430 - val_loss: 0.2011
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - accuracy: 0.9433 - loss: 0.1974 - val_accuracy: 0.9438 - val_loss: 0.1961
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.9456 - loss: 0.1909 - val_accuracy: 0.9455 - val_loss: 0.1906
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - accuracy: 0.9466 - loss: 0.1858 - val_accuracy: 0.9488 - val_loss: 0.1840
```

Program:

```
results = model.evaluate(x_test, y_test, verbose=0)
print('Test loss, Test accuracy:', results)
```

Output:

Test loss, Test accuracy:

[0.2682029604911804,0.9257000088691711]

EXPERIMENT-2

Aim: Design a neural network for classifying movie reviews (Binary Classification) using IMDB (Internet Movie Data Base) dataset.

Discription:

We'll be working with "IMDB dataset", a set of 50,000 highly-polarized reviews from the Internet Movie Database. They are split into 25,000 reviews for training and 25,000 reviews for testing, each set consisting in 50% negative and 50% positive reviews.You should never test a machine learning model on the same data that you used to train it! Just because a model performs well on its training data doesn't mean that it will perform well on data it has never seen, and what you actually care about is your model's performance on new data. Just like the MNIST dataset, the IMDB dataset comes packaged with Keras. It has already been preprocessed: the reviews (sequences of words) have been turned into sequences of integers, where each integer stands for a specific word in a dictionary.

Program:

```
import pandas as pd
import numpy as np
import re
from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
data = pd.read_csv("IMDB_Dataset.csv")
print(data.head())
data.head()
data.tail()
```

Output():

```
Total reviews: 50000
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
```

Program:

```
def clean_text(text):

  text = text.lower()
```

```
    text = re.sub('<.*?>', '', text)

    text = re.sub('[^a-zA-Z]', ' ', text)

    text = re.sub(r'\s+', ' ', text)

    return text.strip()

data['clean_review'] = data['review'].apply(clean_text)
```

Output():

```
                                                   review
0  One of the other reviewers has mentioned that ...
1  A wonderful little production. <br /><br />The...
2  I thought this was a wonderful way to spend ti...
3  Basically there's a family where a little boy ...
4  Petter Mattei's "Love in the Time of Money" is...

                                        clean_review
0  one of the other reviewers has mentioned that ...
1  a wonderful little production the filming tech...
2  i thought this was a wonderful way to spend ti...
3  basically there s a family where a little boy ...
4  petter mattei s love in the time of money is a...
```

Program:

```
X = data['clean_review']

y = data['sentiment'].map({'positive': 1, 'negative': 0})

max_words = 10000

max_len = 200

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(X)
X_seq = tokenizer.texts_to_sequences(X)

X_pad = pad_sequences(X_seq, maxlen=max_len)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=128, input_length=max_len))
model.add(LSTM(128, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
```

Output():

```
Epoch 1/5
563/563 ━━━━━━━━━━━━━━━━━━━━ 327s 572ms/step - accuracy: 0.7206 - loss: 0.5308 - val_accuracy: 0.8353 - val_loss: 0.3906
Epoch 2/5
563/563 ━━━━━━━━━━━━━━━━━━━━ 324s 576ms/step - accuracy: 0.8632 - loss: 0.3418 - val_accuracy: 0.8382 - val_loss: 0.3953
Epoch 3/5
563/563 ━━━━━━━━━━━━━━━━━━━━ 386s 583ms/step - accuracy: 0.8727 - loss: 0.3148 - val_accuracy: 0.8428 - val_loss: 0.3918
Epoch 4/5
563/563 ━━━━━━━━━━━━━━━━━━━━ 377s 575ms/step - accuracy: 0.8874 - loss: 0.2883 - val_accuracy: 0.8460 - val_loss: 0.3963
Epoch 5/5
563/563 ━━━━━━━━━━━━━━━━━━━━ 381s 573ms/step - accuracy: 0.9146 - loss: 0.2200 - val_accuracy: 0.8630 - val_loss: 0.3520
```

Program:

```
y_pred_prob = model.predict(X_test)

y_pred = (y_pred_prob > 0.5).astype(int)

print("Accuracy:", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

Output:

```
Accuracy: 0.8641
              precision    recall  f1-score   support

           0       0.91      0.80      0.85      4961
           1       0.83      0.93      0.87      5039

    accuracy                           0.86     10000
   macro avg       0.87      0.86      0.86     10000
weighted avg       0.87      0.86      0.86     10000
```

EXPERIMENT-3

Aim: Design a neural Network for classifying news wires (Multi class classification) using Reuters dataset.

Description: Reuters dataset: We will work with the Reuters dataset, a set of short newswires and their topics, published by Reuters in 1986. It's a simple, widely used toy dataset for text classification. There are 46 different topics; some topics are more represented than others, but each topic has at least 10 examples in the training set. Like IMDB and MNIST, the Reuters dataset comes packaged as part of Keras.

we will build a model to classify Reuters newswires into 46 mutually exclusive topics. Because we have many classes, this problem is an instance of multiclass classification; and because each data point should be classified into only one category, the problem is more specifically an instance of single-label, multiclass classification. If each data point could belong to multiple categories (in this case, topics), you'd be facing a multilabel, multiclass classification problem.

Program:

```
import numpy as np

import warnings

warnings.filterwarnings("ignore")

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import reuters

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt

(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)

print(len(train_data), len(test_data))

print(train_data[10])

word_index = reuters.get_word_index()

reverse_word_index = {value: key for key, value in word_index.items()}

decoded_newswire = ' '.join([reverse_word_index.get(i - 3, '?') for i in train_data[0]])

print(decoded_newswire)

print(train_labels[10])

def vectorize_sequences(sequences, dimension=10000):
```

```
results = np.zeros((len(sequences), dimension))
for i, sequence in enumerate(sequences):
results[i, sequence] = 1.0
return results
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
one_hot_train_labels = to_categorical(train_labels, 46)
one_hot_test_labels = to_categorical(test_labels, 46)
model = keras.Sequential([
layers.Dense(64, activation='relu', input_shape=(10000,)),
layers.Dense(64, activation='relu'),
layers.Dense(46, activation='softmax')
])
model.compile(
optimizer='rmsprop',
loss='categorical_crossentropy',
metrics=['accuracy']
)
x_val = x_train[:1000]
partial_x_train = x_train[1000:]
y_val = one_hot_train_labels[:1000]
partial_y_train = one_hot_train_labels[1000:]
history = model.fit(
partial_x_train,
partial_y_train,
epochs=20,
batch_size=512,
validation_data=(x_val, y_val)
)
```

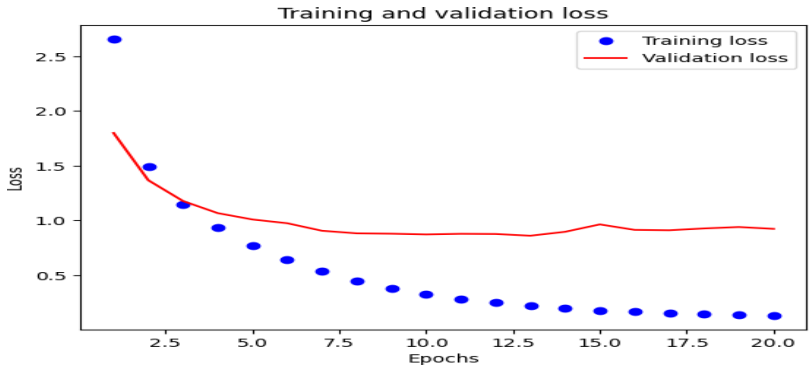
Output:

```
Epoch 1/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 108ms/step - accuracy: 0.3490 - loss: 3.1467 - val_accuracy: 0.6250 - val_loss: 1.8000
Epoch 2/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - accuracy: 0.6673 - loss: 1.6132 - val_accuracy: 0.6900 - val_loss: 1.3678
Epoch 3/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.7370 - loss: 1.1905 - val_accuracy: 0.7440 - val_loss: 1.1779
Epoch 4/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.7926 - loss: 0.9591 - val_accuracy: 0.7650 - val_loss: 1.0674
Epoch 5/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.8241 - loss: 0.7958 - val_accuracy: 0.7710 - val_loss: 1.0095
Epoch 6/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.8506 - loss: 0.6716 - val_accuracy: 0.7780 - val_loss: 0.9752
Epoch 7/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.8799 - loss: 0.5420 - val_accuracy: 0.7990 - val_loss: 0.9064
```

Program:

```
loss = history.history['loss']

val_loss = history.history['val_loss']

epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'bo', label='Training loss')

plt.plot(epochs, val_loss, 'r', label='Validation loss')

plt.title('Training and validation loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.show()
```

Output:

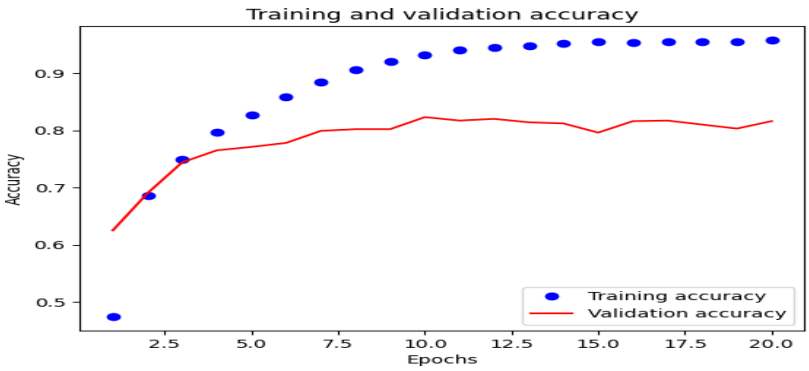


Program:

```
plt.clf()

acc = history.history['accuracy']

val_acc = history.history['val_accuracy']

plt.plot(epochs, acc, 'bo', label='Training accuracy')

plt.plot(epochs, val_acc, 'r', label='Validation accuracy')

plt.title('Training and validation accuracy')
```

```
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Output:



EXPERIMENT-4

Aim: Design a neural network for predicting house prices using Boston Housing Price dataset**.**

Description: This project aims to predict the median value of owner-occupied homes in the Boston area using deep learning techniques. The model is trained on the Boston Housing dataset, which consists of various features such as crime rate, average number of rooms, and accessibility to radial highways. This project demonstrates the effectiveness of deep learning in solving regression problems, specifically in the case of predicting housing prices. Further improvements can be made by exploring different neural network architectures and hyperparameter tuning.

Program:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.datasets import boston_housing

from sklearn.preprocessing import StandardScaler

(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = Sequential([

Dense(64, activation='relu', input_shape=(13,)),

Dense(32, activation='relu'),

Dense(1)  # Output layer for regression

])

model.compile(
optimizer='adam',
loss='mse',
)

 metrics=['mae']
history = model.fit(
X_train, y_train,
```

epochs=100,
batch_size=16,
validation_split=0.1,

verbose=1

)

Output:

```
Epoch 1/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - loss: 578.1310 - mae: 22.1641 - val_loss: 430.0098 - val_mae: 19.7802
Epoch 2/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 458.7945 - mae: 19.7737 - val_loss: 332.0533 - val_mae: 17.1594
Epoch 3/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 368.0462 - mae: 17.2794 - val_loss: 201.5952 - val_mae: 13.0048
Epoch 4/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 211.3327 - mae: 12.6470 - val_loss: 89.7802 - val_mae: 8.0061
Epoch 5/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 113.9162 - mae: 8.2033 - val_loss: 49.3242 - val_mae: 5.9246
Epoch 6/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 69.7240 - mae: 6.3412 - val_loss: 32.5632 - val_mae: 4.5992
Epoch 7/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 34.7501 - mae: 4.6699 - val_loss: 23.2185 - val_mae: 3.7778
Epoch 8/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 35.4608 - mae: 4.2295 - val_loss: 19.8853 - val_mae: 3.4970
Epoch 9/100
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 25.5613 - mae: 3.5841 - val_loss: 18.5867 - val_mae: 3.3493
```

Program:

test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)

print("Test Mean Absolute Error:", test_mae)

Output:

Test Mean Absolute Error: 2.759478807449341

predictions = model.predict(X_test[:5])

print("\nPredicted House Prices:")

print(predictions.flatten())

Output:

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step

Predicted House Prices:
[ 8.611804 18.973236 21.777725 34.182747 25.217524]
```

Program:

print("\nActual House Prices:")

print(y_test[:5])

Output:

Actual House Prices:

[7.2 18.8 19. 27. 22.2]

EXPERIMENT-5

Aim: Build a Convolution Neural Network for MNIST Hand written Digit Classification.

Description:

The MNIST handwritten digit classification problem is a standard dataset used in computer vision and deep learning.Although the dataset is effectively solved, it can be used as the basis for learning and practicing how to develop, evaluate, and use convolutional deep learning neural networks for image classification from scratch. This includes how to develop a robust test harness for estimating the performance of the model, how to explore improvements to the model, and how to save the model and later load it to make predictions on new data.

• How to develop a test harness to develop a robust evaluation of a model and establish a baseline of performance for a classification task.

• How to explore extensions to a baseline model to improve learning and model capacity.

• How to develop a finalized model, evaluate the performance of the final model, and use it to make predictions on new images.

Program:

1. Load libraries

```
import tensorflow as tf

from tensorflow.keras import layers, models

import numpy as np

import matplotlib.pyplot as plt
```

2. Load mnist dataset

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

3. Preprocess data

```
# Normalize

x_train = x_train / 255.0

x_test = x_test / 255.0

# Reshape for CNN (samples, height, width, channels)

x_train = x_train.reshape(-1, 28, 28, 1)

x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode labels

y_train = tf.keras.utils.to_categorical(y_train, 10)

y_test = tf.keras.utils.to_categorical(y_test, 10)
```

4. Building CNN model

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

5. Training the model

```
model.fit(
    x_train,
    y_train,
    epochs=12,
    batch_size=500
)
```

Output:

Epoch 1/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 154s 1s/step- accuracy: 0.0968- loss: 2.4955

Epoch 2/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 201 s 1s/step- accuracy: 0.0957- loss: 2.4752

Epoch 3/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 203 s 1s/step- accuracy: 0.0995- loss: 2.4479

Epoch 4/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 151s 1s/step- accuracy: 0.0984- loss: 2.4262

Epoch 5/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 204 s 1s/step- accuracy: 0.0980- loss: 2.4085

Epoch 6/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 201 s 1s/step- accuracy: 0.0970- loss: 2.3864

Epoch 7/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 202 s 1s/step- accuracy: 0.0982- loss: 2.3699

Epoch 8/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 152s 1s/step- accuracy: 0.0972- loss: 2.3520

Epoch 9/12 120/120 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 201 s 1s/step- accuracy: 0.0975- loss: 2.3324

120/120 ━━━━━━━━━━━━━━━━━━━━ 203 s 1s/step - accuracy: 0.0966 - loss: 2.3151

Epoch 11/12
120/120 ━━━━━━━━━━━━━━━━━━━━ 153s 1s/step - accuracy: 0.0960 - loss: 2.3017

Epoch 12/12
120/120 ━━━━━━━━━━━━━━━━━━━━ 201 s 1s/step - accuracy: 0.0972 - loss: 2.2847

Program:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_accuracy)
```

Output:

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9916 - loss: 0.0284 Test Accuracy: 0.9916

Program:

```
predictions = model.predict(x_test)

predicted_label = np.argmax(predictions[0])

true_label = np.argmax(y_test[0])

print("Predicted Label:", predicted_label)

print("True Label:", true_label)
```

Output:

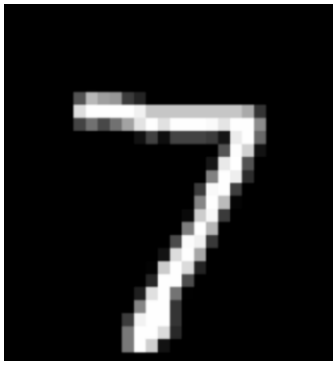
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step Predicted Label: 7

True Label: 7

Program:

```
plt.imshow(x_test[0].reshape(28, 28), cmap="gray")

plt.axis("off")

plt.show()
```

Output:



EXPERIMENT-6

Aim: Build a Convolution Neural Network for simple image (dogs and Cats) Classification.

Description: The Dogs & Cats is a foundational problem for a basic CNN(convolutional neural network) model which involves classifying images as a dog or a cat.The dataset can be used for learning how to develop,evaluate and use convolutional deep learning neural networks for classification of images. This includes how to develop a robust test harness for estimating the performance of the model, exploring improvements for the model by changing the paramters of the model, saving and loading the model to make predicitions on new data.

Program:

```
import matplotlib.pyplot as plt

import tensorflow as tf

import pandas as pd

import numpy as np

import warnings

warnings.filterwarnings('ignore')

from tensorflow import keras

from keras import layers

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Activation, Dropout, Flatten, Dense

from tensorflow.keras.layers import Conv2D, MaxPooling2D

from tensorflow.keras.utils import image_dataset_from_directory

from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img

from tensorflow.keras.preprocessing import image_dataset_from_directory

import os

import matplotlib.image as mpimg

from zipfile import ZipFile

data_path = 'dog-vs-cat.zip'

with ZipFile(data_path, 'r') as zip:

    zip.extractall()

    print('The data set has been extracted.')

fig = plt.gcf()

fig.set_size_inches(16, 16)
```

```
cat_dir = os.path.join('dog-vs-cat-classification/cat')
dog_dir = os.path.join('dog-vs-cat-classification/dog')
cat_names = os.listdir(cat_dir)
dog_names = os.listdir(dog_dir)
pic_index = 210
cat_images = [os.path.join(cat_dir, fname)
              for fname in cat_names[pic_index-8:pic_index]]
dog_images = [os.path.join(dog_dir, fname)
              for fname in dog_names[pic_index-8:pic_index]]
for i, img_path in enumerate(cat_images + dog_images):
    sp = plt.subplot(4, 4, i+1)
    sp.axis('Off')
    img = mpimg.imread(img_path)
    plt.imshow(img)
plt.show()
```

Output:

Program:

4.Splitting dataset:

```
base_dir = 'dog-vs-cat-classification'
train_datagen = image_dataset_from_directory(base_dir,
                                             image_size=(200,200),
```

```
                                    subset='training',
                                    seed = 1,
                                    validation_split=0.1,
                                    batch_size= 32)
test_datagen = image_dataset_from_directory(base_dir,
                                    image_size=(200,200),
                                    subset='validation',
                                    seed = 1,
                                    validation_split=0.1,
                                    batch_size= 32)
```

Output:

Found 25000 files belonging to 2 classes.
Using 22500 files for training.
Found 25000 files belonging to 2 classes.
Using 2500 files for validation.

Program:

```
model = tf.keras.models.Sequential([
layers.Conv2D(32, (3, 3), activation='relu', input_shape=(200, 200, 3)),
layers.MaxPooling2D(2, 2),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D(2, 2),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D(2, 2),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D(2, 2),
layers.Flatten(),
layers.Dense(512, activation='relu'),
layers.BatchNormalization(),
layers.Dense(512, activation='relu'),
layers.Dropout(0.1),
layers.BatchNormalization(),
```

```
layers.Dense(512, activation='relu'),
layers.Dropout(0.2),
layers.BatchNormalization(),
layers.Dense(1, activation='sigmoid')
])
model.summary()
```

Output:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 198, 198, 32)      896

 max_pooling2d (MaxPooling2D  (None, 99, 99, 32)       0
 )

 conv2d_1 (Conv2D)           (None, 97, 97, 64)        18496

 max_pooling2d_1 (MaxPooling  (None, 48, 48, 64)       0
 2D)

 conv2d_2 (Conv2D)           (None, 46, 46, 64)        36928

 max_pooling2d_2 (MaxPooling  (None, 23, 23, 64)       0
 2D)

 conv2d_3 (Conv2D)           (None, 21, 21, 64)        36928

 max_pooling2d_3 (MaxPooling  (None, 10, 10, 64)       0
 2D)

 flatten (Flatten)           (None, 6400)              0

 dense (Dense)               (None, 512)               3277312

 batch_normalization (BatchN  (None, 512)              2048
 ormalization)

 dense_1 (Dense)             (None, 512)               262656

 dropout (Dropout)           (None, 512)               0

 batch_normalization_1 (Batc  (None, 512)              2048
 hNormalization)

 dense_2 (Dense)             (None, 512)               262656

 dropout_1 (Dropout)         (None, 512)               0

 batch_normalization_2 (Batc  (None, 512)              2048
 hNormalization)

 dense_3 (Dense)             (None, 1)                 513
```

Program:

```
model.compile( loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
history = model.fit(train_datagen,epochs=10,validation_data=test_datagen)
```

Output:

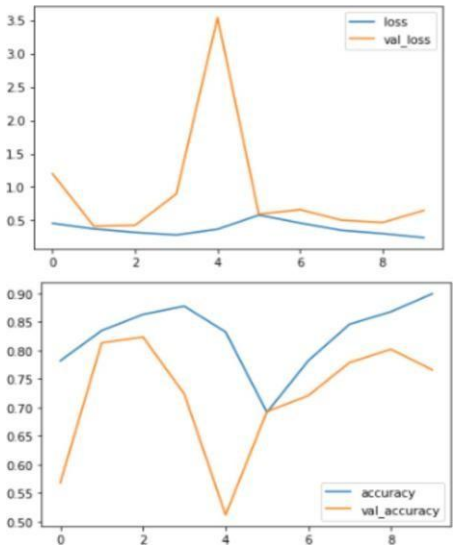
```
Epoch 1/10
704/704 [==============================] - 56s 60ms/step - loss: 0.6837 - accuracy: 0.6174 - val_loss: 0.6314 - val_accuracy: 0.6824
Epoch 2/10
704/704 [==============================] - 42s 59ms/step - loss: 0.5959 - accuracy: 0.6756 - val_loss: 0.7091 - val_accuracy: 0.5256
Epoch 3/10
704/704 [==============================] - 42s 59ms/step - loss: 0.5385 - accuracy: 0.7295 - val_loss: 1.3551 - val_accuracy: 0.5240
Epoch 4/10
704/704 [==============================] - 43s 60ms/step - loss: 0.4555 - accuracy: 0.7871 - val_loss: 0.4511 - val_accuracy: 0.7908
Epoch 5/10
704/704 [==============================] - 43s 60ms/step - loss: 0.4637 - accuracy: 0.7799 - val_loss: 0.7524 - val_accuracy: 0.6628
Epoch 6/10
704/704 [==============================] - 42s 59ms/step - loss: 0.3903 - accuracy: 0.8249 - val_loss: 0.6068 - val_accuracy: 0.7432
Epoch 7/10
704/704 [==============================] - 41s 58ms/step - loss: 0.3705 - accuracy: 0.8323 - val_loss: 0.6982 - val_accuracy: 0.7012
Epoch 8/10
704/704 [==============================] - 42s 59ms/step - loss: 0.3206 - accuracy: 0.8611 - val_loss: 0.3754 - val_accuracy: 0.8360
Epoch 9/10
```

Program:

```
history_df = pd.DataFrame(history.history)
history_df.loc[:, ['loss', 'val_loss']].plot()
```

```
history_df.loc[:, ['accuracy', 'val_accuracy']].plot()

plt.show()
```

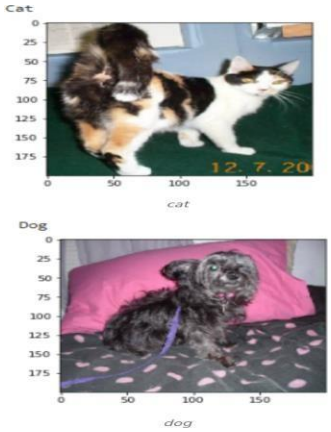
Output:



Program:

```
def predict_image(image_path):

   img = image.load_img(image_path, target_size=(200, 200))

   plt.imshow(img)

   img = image.img_to_array(img)

   img = np.expand_dims(img, axis=0)

   result = model.predict(img)

   print("Dog" if result >= 0.5 else "Cat")

predict_image('dog-vs-cat-classification/cat/cat.320.jpg')

predict_image('dog-vs-cat-classification/dog/dog.5510.jpg')
```

Output:



EXPERIMENT-7

Aim: Use a pre-trained convolution neural network (VGG16) for image classification.

Description: To use a pre-trained VGG16 convolutional neural network for image classification, you would: load the pre-trained VGG16 model, preprocess your new images to match the format expected by the model, then pass the images through the model to obtain predictions about their class labels; essentially leveraging the features learned from a large dataset (like ImageNet) to classify images from your specific domain by replacing the final fully connected layers with ones tailored to your task, a process called transfer learning.

Program:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from glob import glob

loading the directories :

training_dir = '../input/fruits/fruits-360_dataset/fruits-360/Training/'

validation_dir = '../input/fruits/fruits-360_dataset/fruits-360/Test/'

test_dir = '../input/fruits/fruits-360_dataset/fruits-360/test-multiple_fruits/'

image_files = glob(training_dir + '/*/*.jp*g')

valid_image_files = glob(validation_dir + '/*/*.jp*g')

!mkdir ~/.keras

!mkdir ~/.keras/models

!cp ../input/keras-pretrained-models/*notop* ~/.keras/models/

!cp ../input/keras-pretrained-models/imagenet_class_index.json ~/.keras/models/

from keras.models import Model

from keras.layers import Flatten, Dense

from keras.applications import VGG16

#from keras.preprocessing import image

IMAGE_SIZE = [64, 64]

vgg = VGG16(input_shape = IMAGE_SIZE + [3], weights = 'imagenet', include_top = False)

= (64,64,3) as required by VGG

for layer in vgg.layers:

    layer.trainable = False
```

x = Flatten()(vgg.output)

x = Dense(num_classes, activation = 'softmax')(x) # adding the output layer with softmax function as this is a multi label classification problem.

model = Model(inputs = vgg.input, outputs = x)

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

Output:

/opt/conda/lib/python3.6/site-packages/h5py/__init__.py:36: FutureWarning: Conversion of the second argument of issubdtype from `float` to `np.floating` is deprecated. In future, it will be treated as `np.float64 == np.dtype(float).type`.

Program:

from ._conv import register_converters as _register_converters

Using TensorFlow backend.

model.summary()

Output:

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 64, 64, 3)         0
_________________________________________________________________
block1_conv1 (Conv2D)        (None, 64, 64, 64)        1792
_________________________________________________________________
block1_conv2 (Conv2D)        (None, 64, 64, 64)        36928
_________________________________________________________________
block1_pool (MaxPooling2D)   (None, 32, 32, 64)        0
_________________________________________________________________
block2_conv1 (Conv2D)        (None, 32, 32, 128)       73856
_________________________________________________________________
block2_conv2 (Conv2D)        (None, 32, 32, 128)       147584
_________________________________________________________________
block2_pool (MaxPooling2D)   (None, 16, 16, 128)       0
_________________________________________________________________
block3_conv1 (Conv2D)        (None, 16, 16, 256)       295168
_________________________________________________________________
block3_conv2 (Conv2D)        (None, 16, 16, 256)       590080
_________________________________________________________________
block3_conv3 (Conv2D)        (None, 16, 16, 256)       590080
_________________________________________________________________
block3_pool (MaxPooling2D)   (None, 8, 8, 256)         0
_________________________________________________________________
block4_conv1 (Conv2D)        (None, 8, 8, 512)         1180160
_________________________________________________________________
block4_conv2 (Conv2D)        (None, 8, 8, 512)         2359808
_________________________________________________________________
block4_conv3 (Conv2D)        (None, 8, 8, 512)         2359808
_________________________________________________________________
block4_pool (MaxPooling2D)   (None, 4, 4, 512)         0
_________________________________________________________________
block5_conv1 (Conv2D)        (None, 4, 4, 512)         2359808
_________________________________________________________________
block5_conv2 (Conv2D)        (None, 4, 4, 512)         2359808
_________________________________________________________________
block5_conv3 (Conv2D)        (None, 4, 4, 512)         2359808
_________________________________________________________________
block5_pool (MaxPooling2D)   (None, 2, 2, 512)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 2048)              0
_________________________________________________________________
dense_1 (Dense)              (None, 75)                153675
=================================================================
Total params: 14,868,363
Trainable params: 153,675
Non-trainable params: 14,714,688
_________________________________________________________________
```

Program:

from keras.preprocessing.image import ImageDataGenerator

from keras.applications.vgg16 import preprocess_input

training_datagen = ImageDataGenerator(

```
                    rescale=1./255,
                    shear_range=0.2,
                    zoom_range=0.2,
                    horizontal_flip=True,
                    preprocessing_function=preprocess_input)
validation_datagen = ImageDataGenerator(rescale = 1./255,
preprocessing_function=preprocess_input)
training_generator = training_datagen.flow_from_directory(training_dir, target_size =
IMAGE_SIZE, batch_size = 200, class_mode = 'categorical')
validation_generator = validation_datagen.flow_from_directory(validation_dir, target_size =
IMAGE_SIZE, batch_size = 200, class_mode = ‘categorical')
```

Output:

```
Found 37836 images belonging to 75 classes.
Found 12709 images belonging to 75 classes.
Model training:
training_images = 37836
validation_images = 12709
```

Program:

```
history = model.fit_generator(training_generator,
        steps_per_epoch = 10000,
        epochs = 1,
        validation_data = validation_generator,
        validation_steps = 3000)
```

Output:

```
Epoch 1/1
 6483/10000 [==================>.........] - ETA: 24:17 - loss: 0.1204 - acc: 0.9794
```

Program:

```
print ('Training Accuracy = ' + str(history.history['acc']))
print ('Validation Accuracy = ' + str(history.history[‘val_acc']))
```

Output:

```
Training Accuracy = [0.9844328519678907]
Validation Accuracy = [0.9669410271628378].
```

EXPERIMENT-8

Aim: Implement one hot encoding of words or characters.

Description: One-hot encoding is the most common, most basic way to turn a token into a vector. You already saw it in action in our initial IMDB and Reuters examples from chapter 3 (done with words, in our case). It consists in associating a unique integer index to every word, then turning this integer index i into a binary vector of size N, the size of the vocabulary, that would be all-zeros except for the i-th entry, which would be 1.

Of course, one-hot encoding can be done at the character level as well. To unambiguously drive home what one-hot encoding is and how to implement it, here are two toy examples of one-hot encoding: one for words, the other for characters.

Program:

1. One hot encoding for words:

```
 sentence = "i love machine learning”

import numpy as np

words = sentence.split()

vocab = sorted(set(words))

word_to_index = {word: i for i, word in enumerate(vocab)}

one_hot_vectors = {}

for word in words:

    vector = np.zeros(len(vocab), dtype=int)

    vector[word_to_index[word]] = 1

    one_hot_vectors[word] = vector

for word, vec in one_hot_vectors.items():

    print(word, ":", vec)
```

Output:

```
i : [0 1 0 0]

love : [0 0 1 0]

machine : [1 0 0 0]

learning : [0 0 0 1]
```

2.one hot encoding for characters:

```
word = “data"
import numpy as np
chars = sorted(set(word))
char_to_index = {ch: i for i, ch in enumerate(chars)}
one_hot_chars = {}
for ch in word:
    vector = np.zeros(len(chars), dtype=int)
    vector[char_to_index[ch]] = 1
    one_hot_chars[ch] = vector
for ch, vec in one_hot_chars.items():
    print(ch, ":", vec)
```

Output:

```
d : [1 0 0 0]
a : [0 1 0 0]
t : [0 0 0 1]
```

EXPERIMENT-9

Aim: Implement word embeddings for IMDB dataset.

Description: Word embedding is essential in natural language processing with deep learning. This technique allows the network to learn about the meaning of the words. In this post, we classify movie reviews in the IMDB dataset as positive or negative, and provide a visual illustration of embedding.

Today is to train a neural network to find out whether some text is globally positive or negative, a task called sentiment analysis.

The first layer of our neural network will perform an operation called word embedding, which is essential in NLP with deep learning.

The IMDB database: We will work with the IMDB dataset, which contains 25,000 movie reviews from IMDB. Each review is labeled as positive or negative from the rating provided by users together with their reviews.

Program:

```
import tensorflow as tf

from tensorflow import keras

print(tf.__version__)
```

Output:

1.8.0

Program:

```
imdb = keras.datasets.imdb

num_words = 20000

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=num_words)

print("Training examples: {}, labels: {}".format(len(train_data), len(train_labels)))
```

Output:

Training examples: 25000, labels: 25000

Program:

```
print(train_data[0])
```

Output:

[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25, 100, 43,

838, 112, 50, 670, 2, 9, 35, 480, 284, 5, 150, 4, 172, 112, 167, 2, 336, 385, 39, 4, 172, 4536,

1111, 17, 546, 38, 13, 447, 4, 192, 50, 16, 6, 147, 2025, 19, 14, 22, 4, 1920, 4613, 469, 4, 22, 71,

87, 12, 16, 43, 530, 38, 76, 15, 13, 1247, 4, 22, 17, 515, 17, 12, 16, 626, 18, 19193, 5, 62, 386,
12, 8, 316, 8, 106, 5, 4, 2223, 5244, 16, 480, 66, 3785, 33, 4, 130, 12, 16, 38, 619, 5, 25, 124, 51,
36, 135, 48, 25, 1415, 33, 6, 22, 12, 215, 28, 77, 52, 5, 14, 407, 16, 82, 10311, 8, 4, 107, 117, 5952, 15, 256, 4, 2, 7, 3766, 5, 723, 36, 71, 43, 530, 476, 26, 400, 317, 46, 7, 4, 12118, 1029, 13,
104, 88, 4, 381, 15, 297, 98, 32, 2071, 56, 26, 141, 6, 194, 7486, 18, 4, 226, 22, 21, 134, 476, 26,
480, 5, 144, 30, 5535, 18, 51, 36, 28, 224, 92, 25, 104, 4, 226, 65, 16, 38, 1334, 88, 12, 16, 283,
5, 16, 4472, 113, 103, 32, 15, 16, 5345, 19, 178, 32]

Program:

```
len(train_data[0]), len(train_data[1])
```

Output:

(218, 189)

Program:

```
max_len = 500
train_data = keras.preprocessing.sequence.pad_sequences(train_data,
                                                        maxlen=max_len)
test_data = keras.preprocessing.sequence.pad_sequences(test_data,
                                                        maxlen=max_len)
print(train_data.shape)
```

Output:

(25000, 500)

Program:

```
print(train_data[0])
```

Output:

```
[  0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0
```

0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	14	22	16	43	530
973	1622	1385	65	458	4468	66	3941	4	173	36	256
5	25	100	43	838	112	50	670	2	9	35	480
284	5	150	4	172	112	167	2	336	385	39	4
172	4536	1111	17	546	38	13	447	4	192	50	16
6	147	2025	19	14	22	4	1920	4613	469	4	22
71	87	12	16	43	530	38	76	15	13	1247	4
22	17	515	17	12	16	626	18	19193	5	62	386
12	8	316	8	106	5	4	2223	5244	16	480	66
3785	33	4	130	12	16	38	619	5	25	124	51
36	135	48	25	1415	33	6	22	12	215	28	77
52	5	14	407	16	82	10311	8	4	107	117	5952
15	256	4	2	7	3766	5	723	36	71	43	530
476	26	400	317	46	7	4	12118	1029	13	104	88

```
15   297    98    32  2071    56    26   141     6   194
 4   226    22    21   134   476    26   480     5   144
18    51    36    28   224    92    25   104     4   226
38  1334    88    12    16   283     5    16  4472   113
15    16  5345    19   178    32]
```

Experiment-10

Aim: : Implement a Recurrent Neural Network for IMDB movie review classification problem.

Description: The process of building a Sentiment Analysis model using Recurrent Neural Networks (RNNs) to classify movie reviews as positive or negative. Sentiment analysis is a popular Natural Language Processing (NLP) task that involves determining the sentiment or emotion expressed in a piece of text. We will use Python, TensorFlow, and Keras to implement the RNN model and analyze the results.

Recurrent Neural Networks (RNN) are to the rescue when the sequence of information is needed to be captured (another use case may include Time Series, next word prediction, etc.). Due to its internal memory factor, it remembers past sequences along with current input which makes it capable to capture context rather than just individual words.

The IMDB dataset consists of movie reviews from the IMDb website, along with labels indicating whether each review is "positive" or "negative" based on the reviewer's opinion. It is designed for binary sentiment classification, with 25,000 reviews for training and an additional 25,000 for testing. Both the training and testing sets have an equal number of positive and negative reviews, making them balanced for sentiment analysis tasks.

Program:

```
import numpy as np

import pandas as pdimport tensorflow as tf

import re

import matplotlib.pyplot as plt

import seaborn as sns

import pickle

from tensorflow.keras.layers import  SimpleRNN, LSTM, GRU, Bidirectional, Dense,

Embedding

from tensorflow.keras.datasets import imdb

from tensorflow.keras.models import Sequential

vocab_size = 5000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

print(x_train[0])

word_idx = {i: word for word, i in word_idx.items()}

print([word_idx[i] for i in x_train[0]])

print("Max length of a review:: ", len(max((x_train+x_test), key=len)))
```

```
print("Min length of a review:: ", len(min((x_train+x_test), key=len)))
from tensorflow.keras.preprocessing import sequence
x_train = sequence.pad_sequences(x_train, maxlen=max_words)
x_test = sequence.pad_sequences(x_test, maxlen=max_words)
x_valid, y_valid = x_train[:64], y_train[:64]
x_train_, y_train_ = x_train[64:], y_train[64:]
embd_len = 32
RNN_model = Sequential(name="Simple_RNN") RNN_model.add(Embedding(vocab_size,
embd_len, input_length=max_words))
RNN_model.add(SimpleRNN(128,
activation='tanh', return_sequences=False))
RNN_model.add(Dense(1, activation='sigmoid'))
print(RNN_model.summary())
RNN_model.compile(
loss="binary_crossentropy", optimizer='adam', metrics=['accuracy']
)
history = RNN_model.fit(x_train_, y_train_, batch_size=64,
epochs=5, verbose=1,
validation_data=(x_valid, y_valid))
print()
print("Simple_RNN Score---> ", RNN_model.evaluate(x_test, y_test, verbose=0))
plt.title('model_accuracy')
plt.ylabel('accuracy')
plt.xlabel('apoch')
plt.legend(['train','val'],loc='upper left')
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25, 100,
```

43, 838, 112, 50, 670, 2, 9, 35, 480, 284, 5, 150, 4, 172, 112, 167, 2, 336, 385, 39, 4, 172, 4536, 1111, 17, 546, 38, 13, 447, 4, 192, 50, 16, 6, 147, 2025, 19, 14, 22, 4, 1920, 4613, 469, 4, 22, 71, 87, 12, 16, 43, 530, 38, 76, 15, 13, 1247, 4, 22, 17, 515, 17, 12, 16, 626, 18, 2, 5, 62, 386, 12, 8, 316, 8, 106, 5, 4, 2223, 2, 16, 480, 66, 3785, 33, 4, 130, 12, 16, 38, 619, 5, 25, 124, 51, 36, 135, 48, 25, 1415, 33, 6, 22, 12, 215, 28, 77, 52, 5, 14, 407, 16, 82, 2, 8, 4, 107, 117, 2, 15, 256, 4, 2, 7, 3766, 5, 723, 36, 71, 43, 530, 476, 26, 400, 317, 46, 7, 4, 2, 1029, 13, 104, 88, 4, 381, 15, 297, 98, 32, 2071, 56, 26, 141, 6, 194, 2, 18, 4, 226, 22, 21, 134, 476, 26, 480, 5, 144, 30, 2, 18, 51, 36, 28, 224, 92, 25, 104, 4, 226, 65, 16, 38, 1334, 88, 12, 16, 283, 5, 16, 4472, 113, 103, 32, 15, 16, 2, 19, 178, 32]

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json

1641221/1641221 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

['the', 'as', 'you', 'with', 'out', 'themselves', 'powerful', 'lets', 'loves', 'their', 'becomes', 'reaching', 'had', 'journalist', 'of', 'lot', 'from', 'anyone', 'to', 'have', 'after', 'out', 'atmosphere', 'never', 'more', 'room', 'and', 'it', 'so', 'heart', 'shows', 'to', 'years', 'of', 'every', 'never', 'going', 'and', 'help', 'moments', 'or', 'of', 'every', 'chest', 'visual', 'movie', 'except', 'her', 'was', 'several', 'of', 'enough', 'more', 'with', 'is', 'now', 'current', 'film', 'as', 'you', 'of', 'mine', 'potentially', 'unfortunately', 'of', 'you', 'than', 'him', 'that', 'with', 'out', 'themselves', 'her', 'get', 'for', 'was', 'camp', 'of', 'you', 'movie', 'sometimes', 'movie', 'that', 'with', 'scary', 'but', 'and', 'to', 'story', 'wonderful', 'that', 'in', 'seeing', 'in', 'character', 'to', 'of', '70s', 'and', 'with', 'heart', 'had', 'shadows', 'they', 'of', 'here', 'that', 'with', 'her', 'serious', 'to', 'have', 'does', 'when', 'from', 'why', 'what', 'have', 'critics', 'they', 'is', 'you', 'that', "isn't", 'one', 'will', 'very', 'to', 'as', 'itself', 'with', 'other', 'and', 'in', 'of', 'seen', 'over', 'and', 'for', 'anyone', 'of', 'and', 'br', "show's", 'to', 'whether', 'from', 'than', 'out', 'themselves', 'history', 'he', 'name', 'half', 'some', 'br', 'of', 'and', 'odd', 'was', 'two', 'most', 'of', 'mean', 'for', '1', 'any', 'an', 'boat', 'she', 'he', 'should', 'is', 'thought', 'and', 'but', 'of', 'script', 'you', 'not', 'while', 'history', 'he', 'heart', 'to', 'real', 'at', 'and', 'but', 'when', 'from', 'one', 'bit', 'then', 'have', 'two', 'of', 'script', 'their', 'with', 'her', 'nobody', 'most', 'that', 'with', "wasn't", 'to', 'with', 'armed', 'acting', 'watch', 'an', 'for', 'with', 'and', 'film', 'want', 'an'] Max length of a review:: 2697

Min length of a review:: 70

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove
  warnings.warn(
Model: "Simple_RNN"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
None
Epoch 1/5
390/390 ━━━━━━━━━━━━━━━━━━━━ 72s 179ms/step - accuracy: 0.5254 - loss: 0.6916 - val_accuracy: 0.4375 - val_loss: 0.7136
Epoch 2/5
390/390 ━━━━━━━━━━━━━━━━━━━━ 79s 172ms/step - accuracy: 0.6097 - loss: 0.6542 - val_accuracy: 0.6094 - val_loss: 0.6686
Epoch 3/5
390/390 ━━━━━━━━━━━━━━━━━━━━ 83s 176ms/step - accuracy: 0.7049 - loss: 0.5673 - val_accuracy: 0.7500 - val_loss: 0.5588
Epoch 4/5
390/390 ━━━━━━━━━━━━━━━━━━━━ 67s 173ms/step - accuracy: 0.7546 - loss: 0.5090 - val_accuracy: 0.4219 - val_loss: 0.7495
Epoch 5/5
390/390 ━━━━━━━━━━━━━━━━━━━━ 83s 175ms/step - accuracy: 0.6742 - loss: 0.6043 - val_accuracy: 0.7031 - val_loss: 0.6568

Simple_RNN Score---> [0.6161355972290039, 0.6556800007820129]
```